

UNIP

UNIVERSIDADE PAULISTA

Engenharia de Software Ágil Aplicada

Autor: Prof. Edson Quedas Moreno

Colaboradores: Prof. Antonio Palmeira de Araújo Neto
Profa. Christiane Mazur Doi

Professor conteudista: Edson Quedas Moreno

Possui mestrado em Engenharia de Produção com ênfase em Sistemas de Informação (2002) pela Universidade Paulista (UNIP), graduação em Engenharia Eletrônica com ênfase em Computação (1983), pela Faculdade de Engenharia Industrial (FEI), e pós *lato sensu* em Novas Tecnologias de Ensino-Aprendizagem pela Unifitalo (2011). Atua há 40 anos nas áreas de educação e ensino. Desde 1989, exerce as funções de professor e conteudista em cursos de graduação e pós-graduação nas áreas de Negócios, Computação e Informação pelas instituições UNIP e Unifitalo. Foi coordenador de cursos, entre 1989 e 2016, das instituições Unifitalo e Kroton/Anhanguera. Desde 2002, atua como professor, tutor e conteudista pelas instituições UNIP, Unifitalo, Senac, Kroton/Anhanguera, FECAP e no MBA em Engenharia de Software e da Informação da Uninove. Possui mais de 25 anos de experiência na área corporativa.

Dados Internacionais de Catalogação na Publicação (CIP)

M843e Moreno, Edson Quedas.

Engenharia de Software Ágil Aplicada / Edson Quedas Moreno.
– São Paulo: Editora Sol, 2026.

176 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. Software. 2. Normas. 3. Qualidade. I. Título.

681.3

U524.34 – 26

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marília Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. M. Marisa Regina Paixão
Vice-Reitora de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:

Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:

Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:

Prof. Alexandre Ponzetto

Revisão:

Lucas Ricardi
Ingrid Romão
Maitê Donato

Sumário

Engenharia de Software Ágil Aplicada

APRESENTAÇÃO	9
INTRODUÇÃO	12

Unidade I

1 QUALIDADE E PRODUTIVIDADE DE SOFTWARE	15
1.1 Demanda, qualidade e produtividade	15
1.2 Importância da qualidade no desenvolvimento ágil	16
1.2.1 Controle da qualidade: garantia de qualidade de software (GQS) ou software quality assurance (SQA)	17
1.2.2 Custo da qualidade em engenharia de software e metodologias ágeis	21
1.3 Fatores, atributos e métricas da qualidade do software	22
1.3.1 Fator de qualidade	22
1.3.2 Atributo da qualidade	23
1.3.3 Métricas da qualidade	24
2 ENGENHARIA DE SEGURANÇA DE SOFTWARE	28
2.1 Gerenciamento e segurança: análise do risco, plano de contingência e seguro	28
2.2 Gerenciamento de riscos do projeto	31
2.3 Governança em TI	33
2.4 Estratégia para suporte de software	35
2.5 Retorno de investimento (ROI) no ciclo de entrega ágil	36
2.6 Métodos, ferramentas e técnicas (MF&T): melhoria da qualidade de software	36

Unidade II

3 NORMAS E PADRÕES DA QUALIDADE	44
3.1 Processos do ciclo de vida do software: NBR ISO/IEC 12207	45
3.2 Requisitos ergonômicos para trabalhos com computadores: NBR ISO/IEC 9241-11	48
3.3 Avaliação do produto de software: ISO 9126, ISO 14598 e ISO 25000	50
3.3.1 NBR ISO/IEC 9126 – Qualidade do produto da engenharia de software	51
3.3.2 NBR ISO/IEC 14598 – Avaliação do produto de software	54
3.3.3 NBR ISO/IEC 25000 – Guia do SQuaRE	57
3.4 Sistemas da qualidade: ISO 9001 e ISO 90003	58
3.5 Alinhamento das normas da qualidade com metodologias ágeis	62
4 MODELOS DA QUALIDADE DE SOFTWARE	63
4.1 Capability Maturity Model Integration (CMMI®)	64
4.2 SPICE – ISO 15504	65

4.3 MPS.BR	70
4.3.1 Integração com metodologias ágeis.....	71
4.4 Avaliação da qualidade em projetos ágeis.....	71
4.5 Métodos, ferramentas e técnicas (MF&T): metodologia para integração ágil-qualidade com análise SWOT em conformidade com MPS.BR	73

Unidade III

5 PROJETO E CONSTRUÇÃO DO SOFTWARE	83
5.1 Gerenciamento do projeto de software.....	83
5.2 Guia do conhecimento em gerenciamento de projetos PMBOK®	86
5.2.1 Abordagens adaptativas e metodologias ágeis no gerenciamento de projetos.....	93
5.3 Guia de referência para engenharia de software SWEBOK®.....	96
5.3.1 Panorama dos padrões de engenharia de software	99
5.4 Arquiteturas de sistemas: uma abordagem integrada entre PMBOK® e SWEBOK®	100
5.4.1 Arquitetura de sistemas: elementos, tipos e impactos na qualidade.....	101
6 ECOSSISTEMAS DIGITAIS E ENGENHARIA DE SOFTWARE CONECTADA.....	105
6.1 Compreendendo o ecossistema e o negócio digital	105
6.2 Gestão do desenvolvimento ágil em ecossistemas digitais.....	109
6.3 Ecossistemas distribuídos e inteligentes.....	110
6.3.1 Arquitetura distribuída	110
6.3.2 Service-oriented architecture (SOA)	113
6.3.3 Application programming interfaces (APIs).....	115
6.4 Customização de serviços em ecossistemas digitais	118
6.5 Métodos, ferramentas e técnicas (MF&T): desenvolvimento ágil em ecossistemas digitais.....	121

Unidade IV

7 VERIFICAÇÃO & VALIDAÇÃO (V&V)	129
7.1 Verificação do código: depuração e diagnóstico de falhas.....	130
7.2 Testes de software: fundamentos, técnicas e práticas.....	131
7.3 Testes caixa-preta e caixa-branca	132
7.4 Testes unitário, de integração, de aceitação e de regressão.....	136
7.4.1 Teste unitário (unit test)	137
7.4.2 Teste de integração (integration test).....	138
7.4.3 Teste de aceitação (acceptance test)	139
7.4.4 Teste de regressão (regression test).....	140
7.5 Design e comportamento: FDD, DDD e abordagens orientadas a testes	141
7.6 Testes alfa e beta	143
8 GESTÃO DE MUDANÇAS E EVOLUÇÃO DO SOFTWARE	145
8.1 Fundamentos da gestão de mudanças.....	145
8.2 Evolução, integração e tendências do software.....	150
8.2.1 Abordagens ágeis para manutenção e evolução	151

8.2.2 Evolução do software.....	152
8.3 Estratégias estruturais: top-down e bottom-up.....	154
8.3.1 Top-down: integração orientada ao fluxo de valor.....	155
8.3.2 Bottom-up: integração orientada ao componente.....	156
8.4 Gestão de configuração do software.....	157
8.4.1 Versionamento do software.....	158
8.5 Métodos, ferramentas e técnicas (MF&T): controle do código-fonte com GitHub.....	161

APRESENTAÇÃO

A engenharia de software surgiu como resposta à crescente demanda por sistemas cada vez mais complexos, robustos e eficientes. Nos primórdios da computação, o desenvolvimento de software era uma atividade essencialmente artesanal, conduzida de forma pouco estruturada e sem padronização. Com o avanço da tecnologia e a disseminação do software em praticamente todas as áreas da sociedade, tornou-se indispensável a adoção de métodos sistemáticos e organizados para assegurar qualidade, confiabilidade e previsibilidade nos resultados.

A disciplina evoluiu com o propósito de compreender causas, caminhos e soluções que sustentam a construção de produtos de software ao longo de seu ciclo de vida, apresentando conceitos aplicados, exemplos reais, técnicas, papéis profissionais e práticas modernas que moldam a engenharia de software contemporânea. Essa evolução inclui também a incorporação de abordagens ágeis, que introduzem ciclos curtos de entrega, feedback contínuo, integração frequente e adaptação rápida às mudanças, ampliando a capacidade das equipes de produzir software com maior qualidade, valor e previsibilidade.

Por ser uma área da engenharia, sua proposta central é capacitar o aluno com conhecimentos e práticas profissionais essenciais ao projeto e desenvolvimento de sistemas, contemplando tanto métodos tradicionais quanto práticas ágeis, e oferecendo uma base sólida para sua atuação no mercado.

Este livro-texto organiza essa formação em quatro unidades integradas, descritas a seguir.

A unidade I apresenta os fundamentos da qualidade e produtividade de software, destacando como a crescente demanda por sistemas confiáveis e de rápida entrega impulsiona práticas que equilibram eficiência e excelência. Discute-se a importância da qualidade em ambientes de desenvolvimento ágil, nos quais a velocidade das entregas exige mecanismos estruturados de garantia de qualidade (SQA) e práticas consistentes de prevenção de defeitos. São tratados os custos da qualidade em engenharia de software e a forma como metodologias ágeis reduzem desperdícios, ampliam a visibilidade do fluxo de valor e minimizam retrabalho. A unidade também aborda fatores, atributos e métricas da qualidade, diferenciando características globais do produto, propriedades mensuráveis e mecanismos de avaliação que permitem acompanhar a evolução da qualidade ao longo do ciclo de vida.

Ainda nessa unidade, são introduzidos princípios de engenharia de segurança de software, contemplando análise e mitigação de riscos, definição de planos de contingência e papel dos seguros tecnológicos em operações empresariais. O gerenciamento de riscos do projeto é explorado como disciplina essencial para a estabilidade operacional, enquanto a governança de TI é apresentada como estrutura que assegura alinhamento entre tecnologia, negócio e estratégias organizacionais.

Discutiremos também as estratégias de suporte ao software e o retorno de investimento (ROI) aplicado ao ciclo ágil de entrega, demonstrando como equipes podem tomar decisões orientadas por valor. A unidade finaliza com técnicas e ferramentas de melhoria da qualidade, conectando métricas, inspeções e práticas de desenvolvimento contínuo.

A unidade II introduz normas e padrões essenciais à qualidade do software, organizados a partir dos principais referenciais internacionais. Inicia-se pelos processos de ciclo de vida definidos pela ISO/IEC 12207, fundamentais para guiar atividades de aquisição, desenvolvimento, operação e manutenção. São abordados os requisitos ergonômicos da ISO 9241-11, que conectam usabilidade à qualidade de uso e experiência do usuário. Em seguida, são estudadas as normas ISO 9126, 14598 e 25000, que tratam da qualidade de produto, avaliação sistemática e diretrizes do modelo SQuaRE, respectivamente.

A unidade também discute os sistemas de gestão da qualidade ISO 9001 e ISO 90003, orientando sua adaptação ao contexto de software. Além das normas, são explorados diferentes modelos de qualidade, como CMMI®, SPICE/ISO 15504 e MPS.BR, abordando níveis de maturidade, boas práticas e avaliações formais. A integração desses modelos com metodologias ágeis também é analisada, demonstrando como práticas iterativas podem coexistir com estruturas de qualidade robustas. Há também uma discussão sobre a avaliação da qualidade em projetos ágeis, reforçando critérios de inspeção, métricas, revisões contínuas e entregas incrementais. A unidade conclui discutindo técnicas baseadas em análise SWOT aplicadas ao MPS.BR, integrando estratégia organizacional e melhoria contínua no desenvolvimento ágil.

A unidade III trata da relação entre projeto, construção do software e gestão moderna de desenvolvimento. O gerenciamento de projetos é contextualizado a partir do PMBOK®, incluindo áreas de conhecimento, grupos de processos e integração com abordagens adaptativas. As metodologias ágeis aparecem como alternativas que flexibilizam o planejamento e priorizam entregas de valor. O SWEBOOK® é apresentado como guia de referência global para a engenharia de software, fornecendo uma base conceitual sobre requisitos, design, codificação, testes, manutenção e qualidade.

A unidade aprofunda aspectos da arquitetura de sistemas e sua relação com desempenho, segurança, manutenibilidade e alinhamento estratégico. Modelos arquiteturais, elementos estruturais e impactos no ciclo de vida organizacional compõem o panorama técnico. Na sequência, são explorados ecossistemas digitais e sua conexão com a engenharia de software conectada, abordando conceitos de ambientes distribuídos, arquiteturas orientadas a serviços, uso de APIs e a evolução para sistemas inteligentes e integrados. A customização de serviços ganha destaque como diferencial competitivo em ecossistemas dinâmicos, seguida da apresentação de métodos e ferramentas para desenvolvimento ágil aplicado a ambientes digitais complexos.

A unidade IV amplia essa formação ao abordar de forma aprofundada a verificação e validação (V&V), incluindo depuração, diagnóstico de falhas, fundamentos de testes, técnicas caixa-preta e caixa-branca, testes unitários, de integração, aceitação e regressão, além de práticas essenciais, como TDD e testes alfa e beta. Essa unidade também integra os princípios de gestão de mudanças, evolução do software, estratégias estruturais top-down e bottom-up, gestão de configuração, versionamento e ferramentas contemporâneas como o GitHub.

Esse conjunto de quatro unidades oferece uma visão integrada e progressiva da engenharia de software, abrangendo qualidade, segurança, normas, modelos, engenharia do produto, gestão de projetos, ecossistemas digitais e práticas modernas de desenvolvimento. Assim, as quatro unidades convergem para apresentar processos estruturados, metodologias ágeis, padrões internacionais, arquiteturas conectadas, práticas de teste e mecanismos de controle de mudanças que formam uma base robusta, atual e aplicável para profissionais e estudantes que desejam compreender e atuar na engenharia de software moderna.

A linguagem utilizada neste livro-texto é propositalmente técnica, pois reflete o rigor e a precisão exigidos na prática profissional da área. Cada conceito, estudo, pesquisa ou técnica apresentada é resultante de experiências reais em projetos de software que envolvem construção, manutenção, evolução e gestão do ciclo de vida de sistemas modernos. Trata-se, portanto, de um material que conecta teoria e prática, permitindo ao aluno compreender como o conhecimento se manifesta diretamente no mercado.

Embora não esgote todo o vasto campo da engenharia de software, este livro-texto orienta o aluno em direção a essa completude, oferecendo caminhos estruturados de aprofundamento. Ao longo das unidades, você encontrará leituras recomendadas, links úteis, exemplos reais, casos aplicados, referências de ferramentas e tecnologias atuais, além de práticas adotadas em equipes ágeis e em ambientes corporativos de desenvolvimento. Cada seção foi pensada para apoiar sua formação e ampliar sua visão sobre o que significa projetar, desenvolver e evoluir software com qualidade.

Encare este estudo como uma porta de entrada para uma carreira criativa, organizada, responsável, desafiadora e extremamente recompensadora. A engenharia de software é um campo em permanente evolução e este livro-texto foi construído para ajudar você a iniciar essa jornada com conhecimento sólido, olhar crítico e entusiasmo.

Bons estudos!

INTRODUÇÃO

O computador, criado com fins militares na Segunda Guerra Mundial e popularizado a partir de 1947, iniciou uma revolução tecnológica sem precedentes. Com o avanço do transistor e do microprocessador, tornou-se acessível a empresas e residências, e o software, antes secundário, passou a ter papel central como expressão da lógica e criatividade humanas. Essa evolução transformou a sociedade, tornando o software essencial à automação, comunicação e economia.

A engenharia de software surgiu para garantir qualidade e produtividade nesse processo, e nesse contexto a **engenharia de software ágil aplicada** une técnica, gestão e inovação no desenvolvimento de soluções digitais. Em um ambiente de rápidas mudanças, as metodologias ágeis garantem adaptação, colaboração e entregas contínuas de valor. Integradas a normas e modelos de qualidade e segurança, promovem software confiável e alinhado ao mercado.

Este livro-texto apresenta os fundamentos e práticas da engenharia de software sob a ótica ágil, preparando o leitor para aplicar seus conceitos na gestão e construção de sistemas modernos com eficiência e visão estratégica.

A seguir estão os principais tópicos a serem abordados neste livro-texto:

- A **qualidade** e a **produtividade** são fundamentos da engenharia de software. A qualidade assegura que o produto atenda aos requisitos de desempenho, confiabilidade e segurança; já a produtividade otimiza tempo e recursos, mantendo o foco em resultados. As metodologias ágeis reforçam esse equilíbrio, ao promover entregas incrementais e aprimoramento contínuo, enquanto ferramentas de integração e versionamento aumentam a eficiência no ciclo de desenvolvimento.
- A **engenharia de segurança de software** integra-se à engenharia de software para garantir que os sistemas sejam resistentes a falhas e ameaças. Aplicam-se práticas como análise de riscos, controle de acesso e testes de penetração, assegurando a proteção dos dados. Em abordagens ágeis, a segurança é incorporada desde o início do desenvolvimento, evitando vulnerabilidades e retrabalhos.
- As **normas de qualidade** internacionais são essenciais para padronizar processos e mensurar resultados. A NBR ISO/IEC 12207 define o ciclo de vida do software; as ISO 9126, 14598 e 25000 (SQuaRE) tratam da qualidade e avaliação de produtos; e a ISO 9000 orienta a gestão da qualidade organizacional. Todas compõem a base metodológica da engenharia de software, aplicável tanto em modelos tradicionais quanto em abordagens ágeis.
- Os **modelos de qualidade** de maturidade (CMMI®, SPICE – ISO 15504 e MPS-BR) oferecem níveis de evolução e controle de processos, permitindo o aprimoramento contínuo. A engenharia de software os utiliza como referência para medir desempenho organizacional e orientar melhorias. Atualmente, muitas empresas adaptam esses modelos a práticas ágeis, mantendo o rigor da qualidade com a flexibilidade das entregas curtas.

- O **projeto e a construção de software** envolvem a definição de escopo, arquitetura e estratégias de implementação, sustentadas por práticas de gestão e engenharia. O gerenciamento de projetos, pilar da Engenharia de Software, abrange planejamento, custos, equipe e riscos. O PMBOK® reúne boas práticas para gestão de projetos, enquanto o SWEBOK® define áreas específicas da engenharia de software, como requisitos, design, testes e manutenção. Juntos, formam a base para o gerenciamento técnico e organizacional do ciclo de vida do desenvolvimento do software.
- A **gestão de desenvolvimento** alinha os objetivos de negócio às práticas da engenharia de software, garantindo valor real ao produto. Compreender o cliente e o domínio do problema permite definir requisitos e priorizar funcionalidades estratégicas. A SOA estrutura o software em serviços independentes e reutilizáveis, enquanto a customização adapta o sistema às necessidades específicas. Métodos ágeis e ferramentas de apoio asseguram flexibilidade e qualidade no desenvolvimento.
- A **verificação e validação (V&V)** garantem que o software atenda às especificações e funcione corretamente por meio de revisões, auditorias e testes sistemáticos. Incluem depuração de código, testes unitários de integração, aceitação e regressão, além de testes alfa, beta, caixa-preta e caixa-branca. Nas metodologias ágeis, a V&V é contínua e apoiada por testes automatizados, integrando práticas como TDD e BDD, assegurando qualidade constante e entrega de valor durante todo o ciclo de vida do software.
- A **gestão de mudanças no software** garante a evolução contínua do sistema por meio de práticas de controle e adaptação. A manutenção ágil prioriza respostas rápidas e ciclos curtos de entrega, previne defeitos e aumenta a confiabilidade. A gestão de configuração e o versionamento asseguram rastreabilidade e controle das modificações. Práticas como DevOps integram desenvolvimento e operações, promovendo agilidade e qualidade.

A engenharia de software ágil aplicada consolidou-se como o alicerce da era digital, unindo qualidade, produtividade, segurança e inovação. As metodologias ágeis, somadas às normas e padrões internacionais, impulsionam o desenvolvimento de sistemas confiáveis e de alto valor. Este livro-texto convida o leitor a compreender e aplicar esses princípios, desenvolvendo competências essenciais para atuar com excelência na construção do software moderno.

Unidade I

1 QUALIDADE E PRODUTIVIDADE DE SOFTWARE

Além dos demais requisitos, a **qualidade em software** refere-se ao grau em que um produto atende às necessidades e expectativas dos usuários, estando relacionada diretamente aos requisitos não funcionais (RNF) do software.

O software de qualidade apresenta confiabilidade, usabilidade, eficiência, manutenibilidade, portabilidade e segurança. Em outras palavras, ele funciona corretamente, é fácil de usar e manter, e cumpre o propósito para o qual foi desenvolvido.

A **produtividade em software** está relacionada à eficiência do processo de desenvolvimento, isto é, à quantidade de produto (funcionalidades, linhas de código, entregas) gerada em relação aos recursos utilizados (tempo, esforço, equipe e custo). Quanto maior o valor que a equipe entrega em menos tempo e com menos recursos, maior é a produtividade.

A qualidade e a produtividade estão intimamente ligadas e podem parecer conflitantes, mas são interdependentes. No desenvolvimento de software, a busca por alta qualidade e alta produtividade é essencial para a entrega de produtos confiáveis, robustos, eficientes e que agregam valor ao cliente.

Se a equipe tem foco na produtividade (entregar rápido), pode comprometer a qualidade, gerando retrabalho e aumentando custos futuros. Se a equipe tem foco em qualidade excessiva (muitas revisões e controles), a produtividade pode cair devido ao tempo extra gasto.

O equilíbrio ideal é buscar alta produtividade com qualidade sustentável, adotando boas práticas de engenharia de software, automação de testes e melhoria contínua de processos.

1.1 Demanda, qualidade e produtividade

A qualidade de software refere-se à avaliação de um atributo próprio de um determinado componente de software, de modo que ele atenda aos requisitos, seja livre de defeitos e entregue valor ao usuário final. A menos que haja alguma mudança de projeto ou melhoria da tecnologia, um produto de alta qualidade normalmente tem um custo elevado, o que compromete sua competitividade no mercado; por outro lado, a redução da qualidade aumenta o número de defeitos e, consequentemente, o aumento do custo na logística reversa para manutenção, o que leva a um aumento no custo do produto.

Para atender à demanda de um determinado produto, a produção terá que acompanhar e, por vezes, reduzir o custo da produção para que o produto continue competitivo no mercado. Para aumentar a produção, é necessário aumentar o volume produzido. Para isso, deve-se melhorar a produtividade, o que implica em, por exemplo, produzir, no mesmo período ou conforme a necessidade, uma quantidade superior à do período anterior.

A **produtividade de software** mede a eficiência com que a equipe de desenvolvimento transforma requisitos funcionais (RF) em funcionalidades entregues.

Algumas empresas já se adaptaram a essa situação. No entanto, para a grande maioria, o aumento da produção não é acompanhado de qualidade. Algumas empresas enfrentam este desafio: aumentar a produção em menor tempo a custos baixos. Para isso, abrem mão de alguns critérios de qualidade, tais como suporte, inspeção, manutenção, testes, diagnósticos e outros. Consequentemente, o número de defeitos aumenta, encarecendo a logística reversa para suporte e manutenção, elevando, assim, o custo do produto.



Observação

Existe um equilíbrio natural entre demanda e produção. A demanda promove o produto e as vendas, e a produção visa atender à demanda de forma exata, sem desperdícios e a um custo baixo. Esse é o desafio do fornecimento.

1.2 Importância da qualidade no desenvolvimento ágil

A adoção de **metodologias ágeis** (como Scrum, XP, FDD, Kanban e outras), também chamadas de **metodologias adaptativas**, mudou o panorama do desenvolvimento de software, priorizando entregas contínuas e adaptação a mudanças.

Essa agilidade não deve ocorrer em detrimento da excelência técnica. Pelo contrário, a qualidade assume uma posição imperativa e intrínseca no desenvolvimento ágil, atuando como fator habilitador fundamental para a sustentabilidade, a previsibilidade e o sucesso das entregas incrementais.

Diversas metodologias adaptativas, incluindo ágil, adotam um cronograma baseado em fluxo, que não usa um ciclo de vida ou fases. Um dos objetivos é otimizar o fluxo de entregas com base na capacidade de recursos, materiais e outras entradas. Outro objetivo é minimizar o desperdício de tempo e recursos, otimizar a eficiência dos processos e o rendimento das entregas. Projetos que usam essas práticas e métodos geralmente os adotam do sistema de cronograma Kanban usado nas abordagens de elaboração de cronogramas enxutos (lean) e Just-in-Time (JIT) (PMBOK®, 2021, p. 109).

No método ágil, a qualidade não é obtida na etapa final (como no modelo cascata), mas se constrói de forma contínua e integrada. Sem um rigoroso foco na garantia da qualidade do software (GQS) – do inglês, software quality assurance (SQA) –, a velocidade inicial rapidamente se transforma em débito técnico, metáfora para o custo futuro de refatoração e correção que se acumula quando o código de baixa qualidade é priorizado em detrimento do design.

1.2.1 Controle da qualidade: garantia de qualidade de software (GQS) ou software quality assurance (SQA)

A **garantia da qualidade de software (GQS)** é um conjunto sistemático de atividades planejadas e implementadas para garantir que os processos, produtos e artefatos de software estejam em conformidade com os padrões de qualidade estabelecidos pela organização e pelas normas internacionais.

De forma geral, a SQA tem como objetivo assegurar que o software atenda aos requisitos especificados, minimizando defeitos e aumentando a confiabilidade e a satisfação do cliente.

O conceito de qualidade de software vai além da simples verificação da ausência de erros. Ele envolve a adequação do produto ao uso pretendido, sua conformidade com os requisitos do software e a capacidade do processo de desenvolvimento de produzir resultados consistentes e previsíveis.

Assim, o controle da qualidade, dentro do contexto da SQA, atua como um sistema de gestão que acompanha todas as fases do ciclo de vida do software, desde a concepção dos requisitos até a manutenção pós-entrega.

Observe a figura a seguir. Ela demonstra que a SQA abrange: 1. Um processo de SQA; 2. Tarefas específicas da SQA, como revisões técnicas e configuração do software; 3. Testes automatizados e semiautomatizados; 4. Prática efetiva de engenharia de software (métodos e ferramentas); 5. Controle de todos os artefatos de software e as mudanças ocorridas nesses produtos, bem como procedimentos para garantir a conformidade com os padrões de desenvolvimento de software; e 6. Mecanismos de medição e de geração de relatórios.

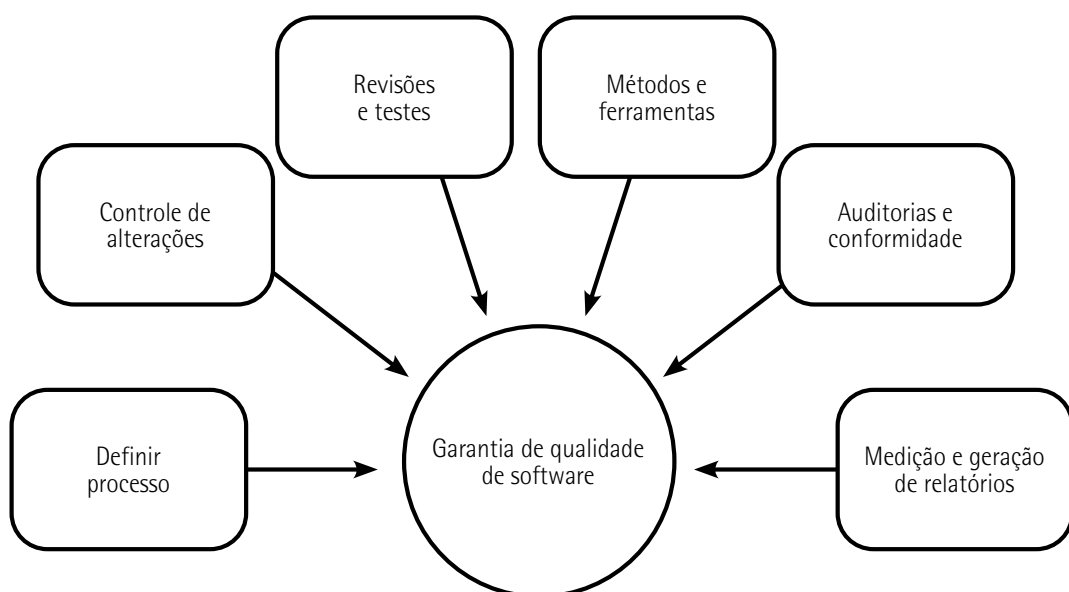


Figura 1 – Garantia de qualidade de software

Fonte: Pressman e Maxim (2021, p. 716).

A qualidade desejada se refere à definição dos RNF do software, determinados para a sua construção. Os RNF correspondem às propriedades de qualidade e às restrições técnicas que influenciam o comportamento global de um sistema de software.

Diferentemente dos RF, que descrevem "o que" o sistema deve fazer, os RNF especificam "como" o sistema deve operar, impactando diretamente sua eficiência, segurança, usabilidade e capacidade de evolução.

Entre os principais RNF a serem avaliados, tendo como referência a ISO 25000, destacam-se:

- **Funcionalidade:** é o grau em que o software atende às especificações estabelecidas nos RF. Uma funcionalidade é caracterizada por um conjunto coeso de operações relacionadas, desenvolvidas de acordo com a mesma base tecnológica e coerentes com a arquitetura do sistema.
- **Confiabilidade:** é a capacidade do software de manter seu desempenho e disponibilidade dentro de limites aceitáveis, mesmo sob condições imprevistas. O software deve ser estável, ter comportamento consistente e apresentar mecanismos de recuperação e tolerância a erros, mesmo diante da imprevisibilidade. Em resumo, deve apresentar um baixo índice de falhas.
- **Usabilidade:** se refere à facilidade de uso, compreensão, atratividade, interação e aprendizado do software. A usabilidade é o principal critério a ser avaliado na interface humano-computador (IHC) e envolve aspectos de experiência do usuário (user experience – UX), acessibilidade e atratividade visual, garantindo uma interação eficiente, agradável e inclusiva.
- **Segurança:** conjunto de práticas, medidas, mecanismos e políticas destinados à proteção do sistema de software contra ameaças, ataques e vulnerabilidades, internos e externos. A segurança contempla a confidencialidade, integridade, disponibilidade e autenticidade das informações, devendo ser incorporada desde as etapas iniciais do ciclo de desenvolvimento.
- **Eficiência:** se refere ao desempenho do software e de suas funcionalidades, com relação a tempo de resposta, uso de recursos computacionais e capacidade de processamento. Além de fornecer resultados corretos (eficazes), a funcionalidade pode ser aprimorada, por exemplo, fornecendo os mesmos resultados "corretos" com redução de tempo e/ou custo, aproveitando a infraestrutura disponível.
- **Manutenibilidade:** corresponde à facilidade de implementar mudanças para atualizações do ambiente operacional, melhoria ou adaptação de funcionalidades a novos contextos, correções e prevenção de falhas. A manutenibilidade é favorecida por boas práticas de codificação, modularização, documentação e versionamento do código-fonte.
- **Portabilidade:** refere-se a quanto um determinado produto de software é portátil para outros ambientes operacionais, ou seja, o quanto pode ser adaptado ou executado em diferentes plataformas, ambientes operacionais ou dispositivos, com o mínimo de esforço técnico. A alta portabilidade contribui para a sustentabilidade e longevidade do produto de software.



Observação

Em resumo, os RNF são determinantes para a qualidade global do produto de software, influenciando tanto na aceitação do usuário final quanto sua viabilidade técnica e econômica ao longo do ciclo de vida.

As eficazes práticas de desenvolvimento de software a seguir melhoram a **relação entre qualidade e produtividade**:

- **Automação de testes:** reduz o tempo de testes manuais e aumenta a cobertura de testes, melhorando a qualidade sem sacrificar a produtividade.
- **Desenvolvimento ágil:** promove entregas frequentes e iterativas, permitindo ajustes rápidos e contínuos que mantêm a qualidade alta e a produtividade constante.
- **Reuso de código:** a reusabilidade é o principal fator de qualidade na engenharia de software. São produzidos componentes reutilizáveis que podem acelerar o desenvolvimento e reduzir a probabilidade de erros, melhorando ambos os aspectos.

Destacam-se também alguns processos e métodos na **SQA**:

- **Verificação e validação (V&V):** é uma das fases críticas do ciclo de desenvolvimento, porque oferece garantias para que o software não apenas satisfaça as necessidades do usuário (**validação**), mas também esteja em conformidade com seus requisitos, especificações e padrões técnicos (**verificação**).

Os **testes de software** são compostos por uma série de técnicas aplicadas durante todo o ciclo de desenvolvimento, que incluem:

- **Teste unitário:** tem foco nos componentes de código isolados.
- **Teste de integração:** verifica e testa a interface entre os módulos e componentes do sistema de software.
- **Teste de sistema:** se refere à validação dos RF e RNF do sistema de software completo.
- **Teste de aceitação do usuário (user acceptance testing – UAT):** assegura que o sistema atenda às necessidades do negócio e seja utilizável pelo cliente.
- **Análise estática e revisão de pares (peer reviews):** são atividades de SQA preventiva, projetadas para identificar e corrigir defeitos já no início do ciclo de desenvolvimento, quando o custo da correção é significativamente baixo. Quando tais defeitos são deixados para depois e um release é liberado para o usuário, isso pode, devido a processos legais e retrabalho em todo o ciclo de desenvolvimento, atingir um milhão de vezes o custo das fases de iniciação.

- A **análise estática** se refere à inspeção do código, da documentação e dos modelos do sistema, para verificar se estão de acordo com os requisitos pré-definidos do software. Essas regras são verificadas antes mesmo da execução do programa.
- As **revisões por pares** são um processo sugerido pela SQA e por diversas metodologias ágeis, em que o trabalho do desenvolvedor, como código-fonte, documentação de requisitos ou modelos de arquitetura, é examinado por um ou mais colegas de trabalho, ou “pares” que possuam competência técnica semelhante.
- As **revisões de código** são revisões formais ou informais, auditorias, walkthroughs e inspeções para identificar problemas precocemente, incluindo inspeções (método estruturado baseado em papéis para detectar defeitos em artefatos, como código e documentação) e o uso de análise estática automatizada como complemento às inspeções manuais, verificando a adesão a padrões de codificação, vulnerabilidades de segurança e complexidade.
- **Gerenciamento de configuração de software (software configuration management – SCM):** o SCM é o processo de identificação, organização e controle sistemático de modificações nos artefatos do sistema durante o ciclo de vida. O SCM se baseia no **controle de versão** com ferramentas e técnicas de versionamento que acompanham o desenvolvimento do software e as entregas aos clientes.

Essas ferramentas têm como objetivo manter um histórico rastreável de todas as alterações, facilitando uma colaboração eficaz em equipes distribuídas, o gerenciamento de branches (ramificações de código) e a capacidade de reverter o sistema a qualquer estado anterior conhecido e estável.

- **Gestão de artefatos e documentação:** artefato de trabalho que garante a rastreabilidade e a manutenibilidade do software. Inclui uma **documentação técnica detalhada** sobre a arquitetura de software, modelo de dados, especificações de requisitos e a documentação da interface de programação de aplicações (application programming interface – API). Em conjunto está o **manual do usuário**, essencial para a usabilidade e aceitação do produto final, idealmente mantido e disponibilizado em formatos eletrônicos pesquisáveis.
- **Gestão de risco e plano de contingência:** essa atividade gerencial identifica, monitora, avalia e mitiga proativamente os riscos que ameaçam a qualidade e a agenda do projeto.
 - A **mitigação de riscos** consiste em definir regras e procedimentos de qualidade (como gateways de qualidade obrigatórios ou limites de cobertura de teste) para evitar a introdução de defeitos críticos.
 - Já o **plano de contingência** se refere à definição de ações de fallback (alternativa de segurança) e recursos a serem mobilizados quando um risco se materializa. O objetivo é minimizar o impacto de falhas, como em planos de reengenharia em implantações de bancos de dados, de forma a garantir a continuidade da produtividade e a estabilidade da entrega.



Saiba mais

As **ferramentas Git**, criadas por Linus Torvalds em 2005, são um exemplo de sistema de controle de versões distribuído (distributed version control system – DVCS).

Saiba mais sobre esse assunto no site a seguir:

COMEÇANDO – uma breve história do Git. *GIT-SCM*, [s.d.]. Disponível em: <https://tinyurl.com/thbnc4h9>. Acesso em: 16 dez. 2025.

1.2.2 Custo da qualidade em engenharia de software e metodologias ágeis

Na engenharia de software, o custo da qualidade (cost of quality – CoQ) representa o conjunto de investimentos, esforços e recursos aplicados para garantir que o produto atenda aos requisitos do software, bem como os custos decorrentes da ausência de qualidade.

De acordo com Pressman e Maxim (2016), esse custo abrange todas as atividades voltadas à prevenção, avaliação e correção de falhas durante o ciclo de vida do software.

Nas metodologias ágeis, o custo da qualidade é entendido como parte do valor entregue ao cliente, ou seja, o equilíbrio entre o esforço investido em qualidade e o retorno percebido em confiabilidade, desempenho e satisfação do usuário final.

Na abordagem do Scrum ou DevOps, a qualidade é continuamente avaliada por meio de iterações curtas, feedbacks constantes, testes automatizados e monitoramento contínuo, que acompanham todo o ciclo de desenvolvimento.

O custo da qualidade pode ser entendido como o preço justo que o cliente esteja disposto a pagar. Esses custos podem ser divididos em:

- **Custos de conformidade:** são os custos necessários para assegurar que o software atenda aos padrões e requisitos de qualidade estabelecidos pela equipe e partes interessadas. Incluem:
 - **Custos de prevenção:** planejamento, revisões técnicas contínuas, projeto, definição de padrões de código, ferramentas de diagnóstico e testes, além de treinamento da equipe de desenvolvimento e dos usuários. Exemplo ágil: refinamento de backlog, sessões de pair programming e retrospectivas de sprint contribuem diretamente para a prevenção de defeitos.
 - **Custos de avaliação:** são os esforços para inspeções intra e interprocessos, produção, modelos de testes e diagnósticos, precisão, manutenção dos equipamentos e plano de mudanças no software. Exemplo ágil: revisões de incremento e testes de regressão automatizados em cada ciclo reduzem o risco de falhas não detectadas.

- **Custos de não conformidade:** representam os custos associados à falta de qualidade, como falhas, retrabalho ou defeitos que são identificados tardiamente, gerando impacto técnico, financeiro ou de reputação.
 - **Custos de falha interna:** revisão de código, engenharia reversa, análise do modo como a falha ocorreu (root cause analysis, ou análise da causa-raiz), refatoração de código, atrasos na entrega e reparação de defeitos. Exemplo ágil: falhas identificadas em testes automatizados durante a integração contínua (pipeline de continuous integration – CI) exigem correção imediata antes da próxima integração.
 - **Custos de falha externa:** solução das queixas, devolução e substituição do produto, incidentes de segurança, manutenção da linha de suporte e serviços da garantia de qualidade e índices de satisfação. Exemplo ágil: após o report de um bug por parte de usuários, em fase de implantação, houve custos adicionais para a correção devido a análises de reversão, afetando, assim, o fluxo de valor.

1.3 Fatores, atributos e métricas da qualidade do software

É importante compreender que a qualidade do software é uma combinação complexa de fatores que variam de acordo com a aplicação e as expectativas dos clientes. Para determinar a qualidade, três conceitos distintos são utilizados: fator de qualidade, atributo de qualidade e métrica de qualidade.

1.3.1 Fator de qualidade

O fator de qualidade é o conjunto de uma ou mais características que influenciam a qualidade do produto de software. Um determinado fator de qualidade agrupa vários atributos específicos a serem implementados no componente de software. O gerenciamento desses atributos determina valores que permitem controlar os níveis de padrões e requisitos aceitáveis para o componente de software.



Observação

Para entender o que é um fator da qualidade, na estrutura do modelo de qualidade da NBR ISO/IEC 9126 são consideradas abordagens sobre requisitos de qualidade externa/interna e a qualidade em uso. Essas abordagens são consideradas fatores de qualidade da NBR ISO/IEC 9126. Cada abordagem compõe um conjunto de atributos do software, chamados na NBR ISO/IEC 9126 de características, tais como funcionalidade, confiabilidade, usabilidade e outros.

Usando o fator de qualidade, é possível detectar áreas problemáticas, permitindo propor soluções e melhorar o processo de software.

Os fatores de qualidade do software podem ser divididos em dois grupos:

- Fatores que podem ser medidos diretamente. Exemplo: defeitos por ponto de função.
- Fatores que podem ser medidos indiretamente. Exemplo: usabilidade ou manutenibilidade.

1.3.2 Atributo da qualidade

O atributo da qualidade representa uma característica particular do produto, passível de ser medida e avaliada. Um ou vários atributos fornecem valores que permitem avaliar um determinado fator de qualidade.

Exemplo de aplicação

Considere um sistema de software de gestão empresarial (ERP) para a logística de suprimentos. Um dos principais atributos da qualidade é a funcionalidade.

Algumas das seguintes métricas são consideradas no sistema de software de gestão empresarial:

- **Usabilidade (RNF 1 – USAB):** medição do índice de satisfação do usuário, de qualquer perfil, nas operações com o sistema de software; nível de implementação de requisitos que atendem às necessidades do usuário. Essas medições podem ser feitas com simples questionamentos (eletrônicos ou não) ao usuário e verificação da quantidade de mudanças requisitadas pelo usuário.
- **Implementação (RNF 2 – IMPL):** acompanhamento do cronograma, observando as metas referentes a prazos e custos. Uma boa ferramenta para isso é o uso de planilhas e gráfico de Gantt.
- **Integração com outras tecnologias (RNF 3 – INTI):** medições de índice de integração da nova tecnologia com a já existente na empresa; grau de portabilidade para outros sistemas de software. Por exemplo: é comum na adaptação em scripts SQL o gerenciamento do banco de dados em plataformas diferentes.
- **Nível de atualizações (RNF 4 – NVAT):** um baixo índice de atualizações indica robustez do projeto de software, o que implica em um menor índice de mudanças a serem implementadas.

No planejamento do controle da qualidade, a distribuição dos atributos da qualidade (os RNF) é feita para cada funcionalidade.

Apesar de em todo software serem considerados todos os RNF, alguns dos atributos da qualidade são característicos de determinados RF. Para cada funcionalidade é definido um grupo de atributos. Veja no quadro 1, a seguir, como isso funciona quando aplicado em um sistema de informações logísticas.

Quadro 1 – Medidas a serem coletadas dos atributos da qualidade

Requisito funcional (RF)	Sistema de informações logísticas (SIL) Requisito não funcional (RNF) – Funcionalidade			
	Atributos da qualidade			
	USAB	IMPL	INTI	NVAT
Cadastros (distribuidor)	X	X	X	X
Cadastros (consumidor)	X		X	X
Comunicação (distribuidor)	X	X	X	
Comunicação (consumidor)		X	X	
Gerenciamento do BD		X	X	X
Controle da conta	X			
Relatórios (distribuidor)	X	X	X	X
Relatórios (consumidor)	X		X	

1.3.3 Métricas da qualidade

Métricas são padrões de medidas dimensionadas para quantificar e qualificar o resultado de um determinado processo, produto ou serviço. A **medida** atende a um determinado padrão que expressa o resultado quantitativo de um atributo específico, seu formato e tipo de dados.

As dimensões das medidas estão relacionadas aos resultados das medições. Por exemplo: comprimento (metro – m); período (segundos – s); massa (grama – g); dígito (bit – b) e diversas outras. Elas podem ser combinadas e associadas para formar um resultado específico. Por exemplo: velocidade em m/s; armazenamento em bytes (1 byte = 8 bits) e taxa de transferência em bps (ou bit/s).

Os dados coletados são analisados por engenheiros de software, comparados com médias anteriores e avaliados para verificar se ocorreu algum desvio no processo de software.

Com as métricas da qualidade, permite-se avaliar dados e informações objetivas sobre o desempenho em processos, atividades e tarefas relacionadas aos atributos da qualidade. Veja as principais métricas da qualidade de software apresentadas no quadro 2 a seguir.

Quadro 2 – Principais métricas de qualidade de software

Métrica	Definição
Acurácia	Capacidade do produto de software de prover, com o grau de precisão necessário, resultados ou efeitos corretos ou conforme acordados
Adaptabilidade	Capacidade do produto de software de ser adaptado para diferentes ambientes especificados, sem necessidade de aplicação de outras ações ou meios além daqueles fornecidos para essa finalidade pelo software considerado
Adequação	Capacidade do produto de software de prover um conjunto apropriado de funções para tarefas e objetivos do usuário especificados
Analisabilidade	Capacidade do produto de software de permitir o diagnóstico de deficiências ou causas de falhas e a identificação de partes a serem modificadas
Apreensibilidade	Capacidade do produto de software de possibilitar ao usuário o aprendizado de sua aplicação
Atratividade	Capacidade do produto de software de ser atraente ao usuário
Auditabilidade	Facilidade no atendimento às normas que pode ser verificado
Autodocumentação	Nível em que o código-fonte fornece documentação significativa
Capacidade	Capacidade do produto de software de ser instalado em ambiente especificado e atender aos requisitos de funcionamento
Coexistência	Capacidade do produto de software de coexistir com outros produtos de software independentes, em um ambiente comum, compartilhando recursos comuns
Confiabilidade	A efetiva realização das funções de um programa em um nível de desempenho e precisão especificadas
Conformidade	Capacidade do produto de software de estar de acordo com normas, convenções ou regulamentações previstas em leis e prescrições similares relacionadas ao atributo do software
Eficácia	Capacidade do produto de software de permitir que usuários atinjam metas especificadas com acurácia e completitude, em um contexto de uso especificado
Eficiência	Quantidade de recursos e códigos de computação para realização da função. Quanto menor a quantidade, maior será o desempenho do software
Estabilidade	Capacidade do produto de software de evitar efeitos inesperados decorrentes de modificações no software
Expansibilidade	Nível em que o projeto arquitetural de dados ou procedimental pode ser estendido
Flexibilidade	Esforço necessário para modificar um programa operacional
Funcionalidade	Capacidade do produto de software de prover funções que atendam às necessidades explícitas e implícitas, quando o software estiver sendo utilizado sob condições especificadas
Generalidade	Âmbito da aplicação potencial dos componentes do programa
Integridade	Diz respeito ao acesso do software ou de dados por pessoas não autorizadas que pode ser controlado
Interoperabilidade	Capacidade do produto de software de interagir com um ou mais sistemas especificados

Métrica	Definição
Mantenabilidade	Esforço necessário para localizar e corrigir erros num programa
Manutenabilidade	Capacidade do produto de software de ser modificado. As modificações podem incluir correções, melhorias ou adaptações do software devido a mudanças no ambiente e nos seus requisitos ou especificações funcionais
Maturidade	Capacidade do produto de software de evitar falhas decorrentes de defeitos no software
Modificabilidade	Capacidade do produto de software de permitir que uma modificação especificada seja implementada
Modularidade	Diz respeito à independência funcional dos componentes do programa
Precisão	Nível de precisão dos cálculos e do controle que pode ser verificado
Operacionalidade	Capacidade do produto de software de possibilitar ao usuário operá-lo e controlá-lo
Produtividade	Capacidade do produto de software de permitir que seus usuários empreguem quantidade apropriada de recursos em relação à eficácia obtida, em um contexto de uso especificado
Portabilidade	Capacidade do produto de software de ser transferido de um ambiente operacional para outro
Rastreabilidade	Diz respeito à habilidade de rastrear uma representação de projeto ou componente real do programa
Recuperabilidade	Capacidade do produto de software de restabelecer seu nível de desempenho especificado e recuperar os dados diretamente afetados no caso de uma falha
Reutilização	Quanto de um programa ou parte dele pode ser reutilizado em outras aplicações
Satisfação	Capacidade do produto de software de satisfazer usuários, em um contexto de uso especificado Nota: satisfação é a resposta do usuário à interação com o produto e inclui atitudes relacionadas ao uso do produto
Segurança	Capacidade do produto de software de apresentar níveis aceitáveis de riscos de danos às pessoas, aos negócios, da propriedade, do ambiente, de proteger informações e dados, de modo que pessoas ou sistemas não autorizados não possam lê-los nem modificá-los e que não seja negado o acesso às pessoas ou sistemas autorizados
Testabilidade	É necessário testar um programa para sua validação, quando criado ou modificado, a fim de garantir que realize a função esperada
Usabilidade	Capacidade do produto de software de ser compreendido, aprendido, operado e atrativo ao usuário, quando usado sob condições especificadas
Utilização	Esforço necessário para aprender, operar, preparar entradas e interpretar saídas de um programa

Fonte: Pressman e Maxim (2021), ABNT (1998) e ABNT (2003).

Exemplo de aplicação

Na sequência do exemplo anterior e com base no quadro 2, determine as métricas da qualidade do requisito funcional gerenciamento do BD.

Para determinar as métricas da qualidade são considerados os atributos da qualidade do requisito funcional gerenciamento do BD: implementação (IMPL), integração com outras tecnologias (INTI) e nível de atualizações (NVAT). Veja como determinar e especificar métricas da qualidade:

- **Implementação (IMPL):** medida direta de produção, em períodos de dias (d.), comparada com as metas definidas no cronograma. No gerenciamento do BD, pode-se considerar como metas das fases de implementação: arquitetura da informação (10 d.), especificação (5 d.), scripts de BD (15 d.), integração (5 d.) e testes e diagnósticos (10 d.).
 - **Integração com outras tecnologias (INTI):** podem ser efetuadas medidas indiretas, tais como medidas de portabilidade do BD, que definem na prática os sistemas operacionais e outros gerenciadores de BDs em que o software irá operar. Outras possíveis medidas de integração com tecnologias variadas são medidas diretas sobre o índice de falhas na interface com outras arquiteturas de dados. Falhas nos cálculos dos resultados obtidos, como inconsistência no tratamento de casas decimais, campos vazios não identificados ou a omissão de resultado esperados, medidas de taxa de transferência da rede e do throughput do sistema, afetam diretamente a análise de desempenho.
 - **Nível de atualizações (NVAT):** são medidas diretas de versionamento do software que indicam o índice de mudanças no código-fonte. São definidas as versões que acompanham o desenvolvimento do software e as versões liberadas ao cliente. O número de mudanças em um determinado período pode indicar instabilidade do software.
-

Práticas eficazes de desenvolvimento de software impactam diretamente as métricas de qualidade, servindo como base objetiva para avaliar a relação entre qualidade e produtividade.

A automação de testes contribui para o aumento da cobertura de testes e, consequentemente, a redução da densidade de defeitos e a diminuição do tempo de detecção de falhas, refletindo maior confiabilidade do produto.

O desenvolvimento ágil, por meio de ciclos curtos e entregas frequentes, favorece o acompanhamento contínuo de métricas como lead time, taxa de retrabalho e defeitos por iteração, possibilitando ajustes rápidos que preservam a qualidade ao longo do tempo.

O reuso de código influencia positivamente métricas de manutenibilidade, complexidade e custo de manutenção, uma vez que componentes reutilizáveis, previamente testados e consolidados, reduzem a ocorrência de erros e aceleram o desenvolvimento.



Lembrete

Em resumo, as métricas de qualidade evidenciam que a aplicação dessas práticas sustenta um equilíbrio consistente entre qualidade do software e produtividade da equipe.

2 ENGENHARIA DE SEGURANÇA DE SOFTWARE

A engenharia de segurança de software planeja, desenvolve e mantém sistemas seguros e confiáveis, protegendo dados e funcionalidades contra falhas, vulnerabilidades e ataques.

Tem como objetivo garantir a integridade, confidencialidade, autenticidade e disponibilidade das informações ao longo de todo o ciclo de vida do software, envolvendo a aplicação de princípios de segurança, desde a fase de requisitos até a de operação, suporte e manutenção.

As técnicas aplicadas fornecem práticas como análise de ameaças, modelagem de riscos, testes de penetração e criptografia. O domínio do sistema mostra um cenário de crescente digitalização e ameaças cibernéticas, e, em resposta, a engenharia de segurança busca o desenvolvimento de sistemas resilientes, capazes de suportar incidentes e manter a confiança dos usuários.

Assim, a engenharia de segurança de software integra a qualidade técnica e a proteção da informação, promovendo soluções tecnológicas seguras, éticas e sustentáveis.

2.1 Gerenciamento e segurança: análise do risco, plano de contingência e seguro

A produção e a operação de software na sociedade contemporânea o posicionam como um ativo crítico, cuja falha ou comprometimento pode gerar perdas financeiras, danos reputacionais e interrupções de serviço de magnitude sistêmica.

Dessa forma, o gerenciamento de risco (GR), o plano de contingência (PC) e a seguridade (seguro) aplicados ao software envolvem práticas sistemáticas que visam identificar, avaliar e mitigar riscos que possam comprometer a qualidade, a continuidade e a confiabilidade dos sistemas computacionais, para a sustentabilidade operacional e a proteção dos ativos de informação.

O **risco** é conceituado como "um evento ou condição incerta que, se ocorrer, tem um efeito positivo ou negativo sobre ao menos um dos objetivos a ser cumprido" (PMBOK®, 2017, p. 397), ou como a possibilidade de acontecer algo que ameace seus objetivos, que pode ser interpretado como a probabilidade de perigo, uma ameaça física para o homem ou para o meio em que se encontra, ou a probabilidade de insucesso em função de um acontecimento ou incidente que acarrete indenização.

A **análise do risco** é o primeiro passo no processo de gerenciamento da segurança. Ela consiste na identificação, categorização e avaliação das ameaças e vulnerabilidades que podem afetar o software, considerando tanto aspectos técnicos (falhas de código, ataques cibernéticos, indisponibilidade de servidores) quanto organizacionais (erros humanos, falhas de comunicação, requisitos mal definidos).

Essa análise deve ser conduzida com base em métodos quantitativos e qualitativos, permitindo estimar a probabilidade de ocorrência e o impacto potencial de cada risco.

No desenvolvimento ágil, a análise de risco é uma atividade contínua, geralmente integrada ao planejamento da sprint e à gestão de backlog. No quadro a seguir são apresentados alguns dos fatores de análise do risco no desenvolvimento de software. O objetivo é proteger a capacidade da equipe de entregar valor de forma rápida e sustentável.

Quadro 3 – Elementos de análise do risco na engenharia de software ágil

Elementos	Definições
Ameaça	Qualquer evento, pessoa ou condição que possa causar um resultado indesejado ou comprometer um artefato ou o processo de software. Exemplo: atraso no fornecimento de uma API de fornecedor terceirizado
Probabilidade	É a chance de uma ameaça se concretizar durante o desenvolvimento de uma funcionalidade, sprint ou o comprometimento com o ciclo de entregas. Exemplo: há grande probabilidade de que a implementação de um novo código contenha regressões, se testes unitários não forem concluídos
Impacto	É a medida do dano ou custo (em tempo, dinheiro, reputação ou funcionalidade) que a ocorrência do risco pode acarretar ao produto de software, ao desempenho da equipe e/ou ao compromisso de entregas. Exemplo: impacto da mudança na ocorrência de uma falha de software que impede a implantação do software
Incerteza	O grau de incerteza é medido de acordo com a falta de informações claras sobre um determinado requisito, uma dependência técnica ou a viabilidade de uma solução. Nas metodologias ágeis, isso é frequente e conhecido como spike (em inglês: atividade de pesquisa ou experimentação técnica), com objetivo de converter a incerteza em informação
Ação alternativa	É a estratégia de mitigação ou plano de contingência (rollback, que significa ação corretiva) definido para neutralizar ou reduzir o risco. Exemplo: implementar um recurso de fallback (recuo) para serviços de terceiros, ou refatorar o código para reduzir o débito técnico



Observação

Na análise do risco deve-se considerar também a questão do não determinístico, como o erro humano no uso do sistema, que pode causar possíveis falhas. Por exemplo: um operador registra alguma ação de configuração, desabilitando o acesso a alguns documentos. Isso pode comprometer erroneamente as operações de algum outro operador, que fica impossibilitado de registrar a validação de algum documento.

Após a análise do risco, estabelece-se o **plano de contingência**, um documento estratégico que define ações preventivas e corretivas para garantir a continuidade do serviço e a recuperação dos sistemas após incidentes. Inclui rotinas de backup e recuperação de dados, estratégias de redundância de servidores, planos de resposta a incidentes de segurança e mecanismos de failover (processo automático de transferência de operações e dados entre servidores), para minimizar o tempo de inatividade e garantir a continuidade dos serviços. Trata-se de práticas de DevOps e de integração contínua/entrega contínua, que facilitam a rápida correção de falhas, com objetivo de reduzir a exposição a riscos.

Na figura a seguir, é apresentada uma arquitetura para a operação de failover. Observe nos estereótipos a mudança de endereços que ocorre de forma automática. O SERVIDOR 1 <<IP>> está na frente do usuário, e, de forma sincronizada, o SERVIDOR 2 – Mirroring <<IP:2>> está sendo atualizado com os dados do SERVIDOR 1 <<IP>>. Se o SERVIDOR 1 <<IP>> falhar, ele será desabilitado e, automaticamente, o SERVIDOR 2 <<IP:2>> assume a frente do usuário na rede, até que o sistema seja estabilizado e retorne às operações normais.

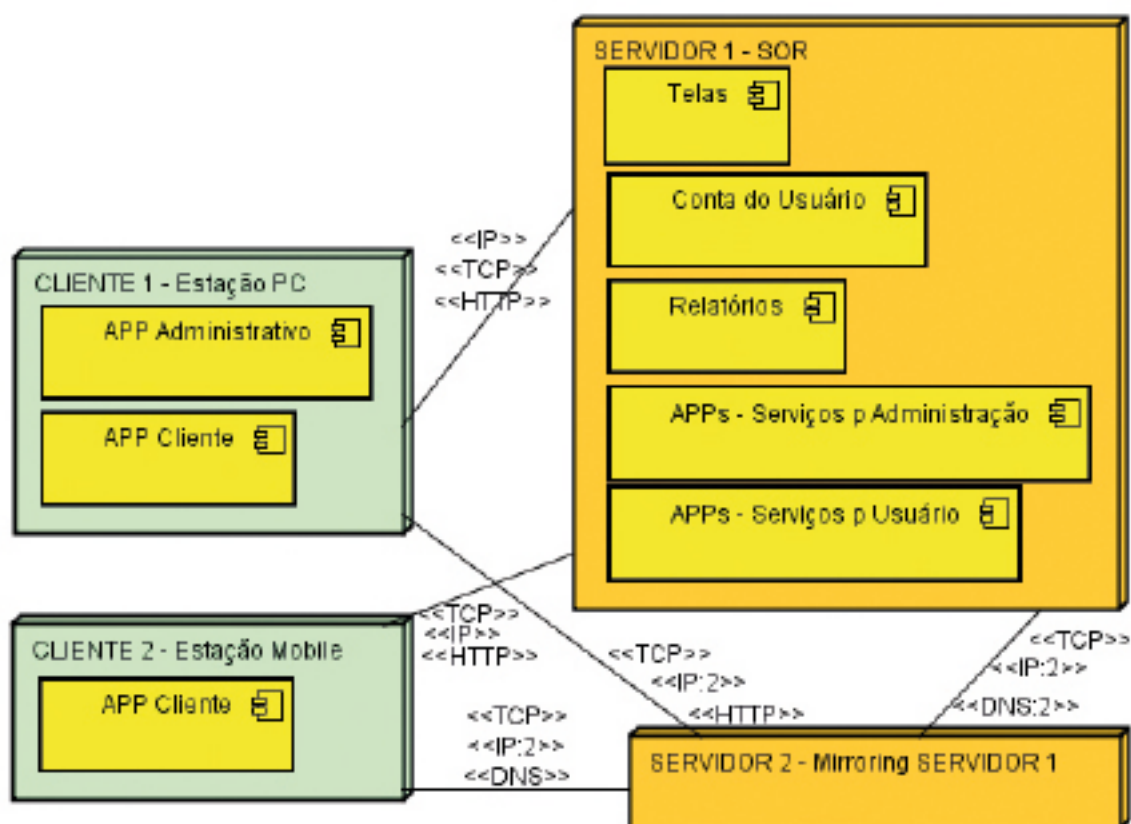


Figura 2 – Arquitetura com espelhamento (mirroring) de servidor para operação de failover

O **seguro** aplicado a software (ou seguro cibernético) surge como uma ferramenta financeira de mitigação de risco, destinada a cobrir prejuízos decorrentes de incidentes de segurança, como vazamento de dados, interrupção de serviços, ataques de ransomware e falhas de infraestrutura.

A modalidade de seguro é cada vez mais adotada por empresas que buscam reduzir os impactos econômicos e legais de eventos imprevistos, funcionando como uma extensão do plano de contingência e do controle de riscos corporativos.

Em resumo, a aplicação conjunta dessas práticas reforça o princípio de que segurança e qualidade são dimensões inseparáveis da engenharia de software moderna, sendo o gerenciamento de riscos uma competência necessária para o desenvolvimento sustentável, ético e confiável de sistemas de software.

O gerenciamento e a segurança de software formam um ciclo contínuo de planejamento, prevenção, resposta e aprendizado, sustentado por políticas claras, auditorias regulares e cultura organizacional voltada à segurança.

A integração entre análise de risco, plano de contingência e seguro garante não apenas proteção técnica, mas também um alto controle estratégico da organização, permitindo que ela mantenha sua operação mesmo diante de falhas, ameaças ou incidentes críticos.

2.2 Gerenciamento de riscos do projeto

O **risco** pode ser conceituado como a combinação da probabilidade da ocorrência de um evento e de suas consequências sobre os objetivos do projeto. Em termos quantitativos, o risco pode ser representado pela relação:

$$\text{Risco} = \text{Probabilidade (P)} \times \text{Impacto (I)}$$

- **Probabilidade (P):** o quanto é provável que o risco ocorra (por exemplo: baixa, média ou alta).
- **Impacto (I):** quão severas serão as consequências, caso o risco ocorra (por exemplo: pequeno, moderado ou crítico).

As equipes de projeto procuram maximizar os riscos positivos (oportunidades) e diminuir a exposição a riscos negativos (ameaças). As ameaças podem resultar em problemas como atraso, excesso de custos, falha técnica, queda de desempenho ou perda de reputação. As oportunidades podem levar a benefícios como tempo e custo reduzidos, desempenho aprimorado, maior participação, ou melhor reputação no mercado (PMBOK®, 2021, p. 65).

Nos modelos prescritivos (cascata, espiral e outros), o gerenciamento do risco é aplicado em fases específicas do ciclo de desenvolvimento, enquanto nas metodologias ágeis (Scrum, XP, Kanban, FDD e outros), o monitoramento e a mitigação de riscos ocorrem de forma iterativa, ou seja, promovem ajustes rápidos ao longo do ciclo de desenvolvimento da funcionalidade ou da sprint.

Em uma visão mais ampla, o PMBOK® (2017) apresenta os processos de gerenciamento dos riscos do projeto em um framework, conforme a figura a seguir. Esse framework organiza processos discretos com interfaces bem definidas. Quando esses processos não são adequadamente gerenciados, os riscos podem desviar o projeto daquilo que foi originalmente planejado.

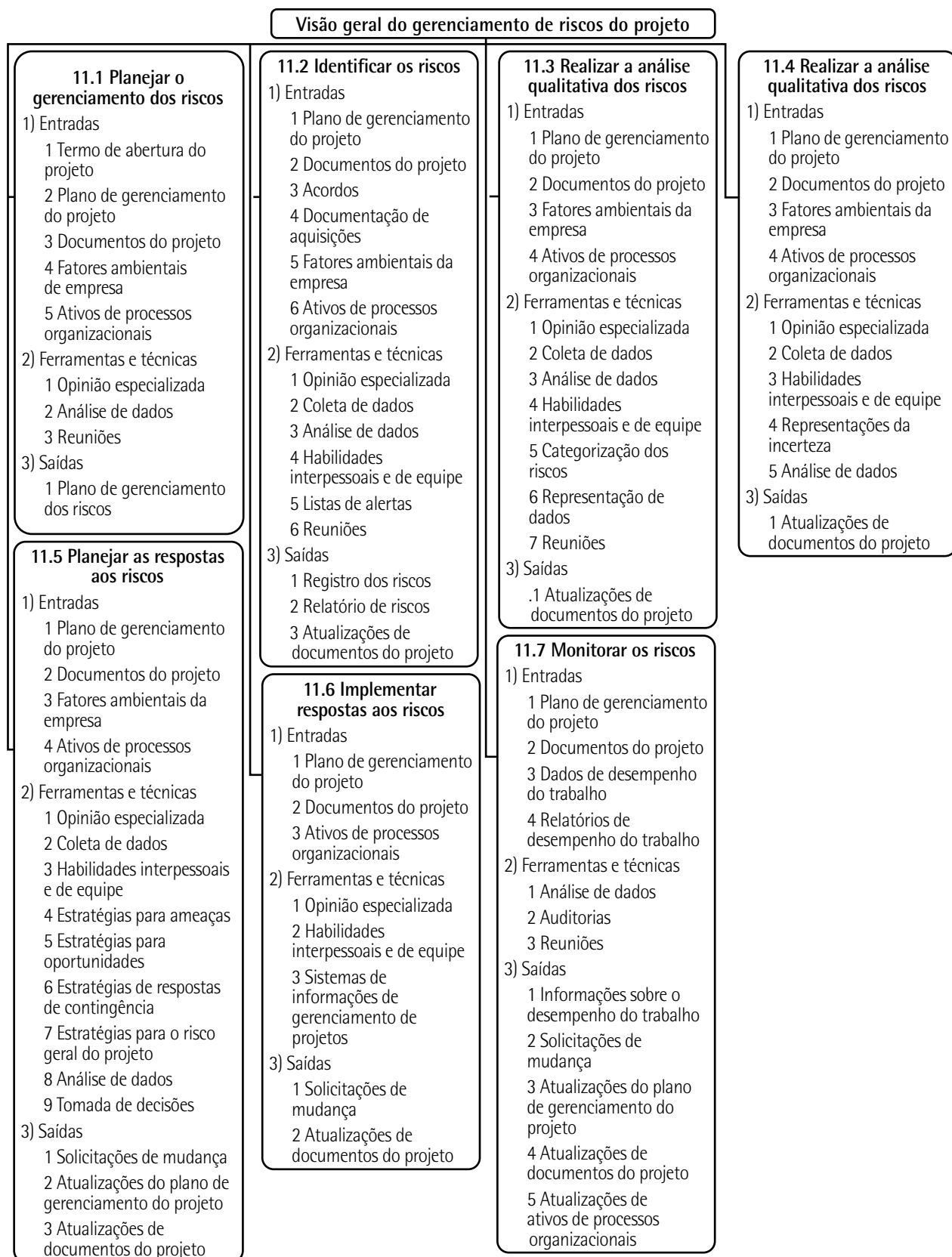


Figura 3 – Visão geral do gerenciamento dos riscos do projeto

Entre as boas práticas do PMBOK® aplicadas às metodologias ágeis, destacam-se os principais **processos de gerenciamento dos riscos do projeto**:

- 1) **Planejar o gerenciamento do risco**: definir como as atividades de risco serão conduzidas no projeto, considerando também a dinâmica das metodologias ágeis.
- 2) **Identificar os riscos**: levantar riscos técnicos, de negócio e de processo por meio de reuniões de planejamento, histórias de usuário (user story) e retrospectivas.
- 3) **Realizar análise qualitativa e quantitativa dos riscos**: priorizar riscos (baixo, médio e alto), conforme seu impacto (pequeno, moderado ou crítico) nas mudanças e/ou nas entregas de curto prazo.
- 4) **Planejar as respostas aos riscos**: definir estratégias como spikes técnicos, automação de testes e pair programming para mitigar incertezas.
- 5) **Implementar respostas aos riscos**: aplicar as ações planejadas de mitigação e contingência durante as iterações. Em um ambiente ágil, isso ocorre durante as sprints, com a equipe ajustando atividades, prioridades e recursos.
- 6) **Monitorar os riscos**: adotar em reuniões a prática de revisões contínuas e periódicas sobre os riscos, promovendo aprendizado contínuo e atualização do risk backlog (lista organizada e priorizado de riscos).



Saiba mais

O Programa de Gestão de Riscos do Microsoft 365 identifica, avalia e gere riscos para o Microsoft 365. A Microsoft identifica e resolve proativamente riscos que podem afetar sua infraestrutura de serviços, seus clientes, seus dados e sua confiança. Saiba mais em:

PROGRAMA de gerenciamento de riscos do Microsoft 365. *Microsoft Ignite*, 29 set. 2025. Disponível em: <https://tinyurl.com/4u3ked8z>. Acesso em: 16 dez. 2025.

2.3 Governança em TI

A governança em tecnologia da informação (TI) se baseia em um conjunto de estratégias, políticas para uso da tecnologia, estruturas e processos decisórios com o intuito de apoiar e expandir os objetivos estratégicos e operacionais da organização.

Frameworks como **COBIT 2019** e **ITIL v4** desempenham papéis fundamentais, fornecendo diretrizes para a governança e a gestão de serviços de TI.

O **COBIT 2019** é uma evolução do COBIT 5 com atualizações significativas, como fatores de design que permitem a personalização do sistema de governança de TI. O Control Objectives for Information and Related Technologies (COBIT) é um framework para **gestão e controle de ambientes de TI empresariais**, com foco na governança corporativa, alinhando as estratégias de negócios com as estratégias de TI para maximizar o valor dos investimentos tecnológicos.



Saiba mais

Information Systems Audit and Control Association (ISACA®) é uma das principais organizações internacionais voltadas à governança, auditoria, segurança e gestão de riscos em TI. É uma associação profissional global sem fins lucrativos, fundada em 1969, com o objetivo de fornecer padrões, certificações e boas práticas em diversas áreas da tecnologia. Para mais informações, acesse o site da ISACA®:

Disponível em: <https://www.isaca.org/>. Acesso em: 16 dez. 2025.

O **ITIL v4**, por sua vez, enfatiza a **gestão de serviços de TI** com foco na entrega de valor e na melhoria contínua. O Information Technology Infrastructure Library (ITIL) integra formas de trabalho como lean, metodologias ágeis e DevOps, permitindo maior flexibilidade e adaptação às mudanças organizacionais.



Saiba mais

ITIL é o padrão global para transformar estratégia baseada na experiência do cliente por meio da transformação digital, aproximando negócios, desenvolvedores e gerentes de serviços. Acesse o site da ITIL:

Disponível em: <https://www.ital.com/>. Acesso em: 16 dez. 2025.

Em vez de estruturas hierárquicas rígidas, nas **metodologias ágeis** são aplicados modelos de governança leve, que equilibram flexibilidade e conformidade para que as equipes de desenvolvimento trabalhem com liberdade para inovar sem comprometer a segurança, a rastreabilidade e a qualidade dos processos.

As metodologias ágeis não se limitam apenas à conformidade e ao controle das fases do desenvolvimento. São instrumentos dinâmicos de gestão estratégica que possibilitam inovação, eficiência e resiliência organizacional, buscando o alinhamento das estratégias do negócio com as estratégias da tecnologia da informação para atingir as metas corporativas de forma contínua.

A abordagem ágil de governança incentiva práticas como:

- **Gestão descentralizada de decisões**, em que as equipes têm autoridade para escolher tecnologias, arquiteturas e métodos dentro de diretrizes corporativas.
- **Integração contínua entre áreas de negócio e tecnologia**, promovendo alinhamento estratégico em tempo real.
- **Monitoramento iterativo de riscos, métricas e resultados**, por meio de frameworks como DevOps, Scrum e Kanban, que favorecem a transparência e o aprendizado contínuo.
- **Entrega incremental de valor**, garantindo que os investimentos em TI gerem benefícios tangíveis e mensuráveis ao negócio.

2.4 Estratégia para suporte de software

O suporte de software não é apenas uma necessidade reativa para corrigir problemas, mas um componente estratégico vital para o sucesso e a longevidade de qualquer aplicação ou sistema de software.

No mundo digital, no qual a tecnologia está no coração das operações de negócio, garantir que o software funcione de maneira eficiente e ininterrupta é crucial. A estratégia do suporte de software deve ser bem definida, indo muito além da simples resolução de falhas ou soluções "apaga incêndio". Ela deve englobar a gestão proativa de riscos, a otimização da experiência do usuário (UX) e a manutenção da satisfação do cliente a longo prazo.

Desenvolver uma estratégia de suporte envolve planejar a infraestrutura de atendimento, que pode variar desde canais de autoatendimento (como bases de conhecimento e FAQs) até equipes especializadas para suporte de nível 1, 2 e 3, determinados de acordo com o tamanho, complexidade e qualidade exigida.

É essencial estabelecer métricas de desempenho claras, como tempo médio de resposta, tempo de resolução e índice de satisfação do cliente, de forma a manter um monitoramento eficaz no suporte, com o objetivo de assegurar a confiabilidade do software e construir uma relação de confiança com os usuários.

As estratégias de suporte de software que combinam **engenharia de segurança de software e metodologias ágeis** buscam equilibrar resposta rápida a incidentes com proteção contínua dos sistemas. O suporte não deve ser apenas reativo (corrigir erros), mas proativo e resiliente, reduzindo riscos e fortalecendo a segurança em cada ciclo de entrega.

Sommerville (2016), nos capítulos 13 e 14 de sua obra *Software engineering*, apresenta alguns princípios referentes à integração da engenharia de segurança com metodologias ágeis, mostradas no quadro a seguir.

Quadro 4 – Princípios de integração da engenharia de segurança e metodologias ágeis

Princípio	Objetivo	Prática
Segurança tratada como RNF	A segurança é tratada como parte da qualidade do software	Definir histórias de usuário com critérios de segurança (security stories)
Resiliência e continuidade	O sistema deve suportar falhas (bugs) e ataques sem perda crítica	Implementar mecanismos de rollback (reverter), redundância de código e dados e logs de auditoria
Iteratividade e resposta rápida	Entregas curtas favorecem correções e reforços de segurança contínuos	Aplicar DevSecOps e pipelines de CI/CD com testes de segurança automatizados
Cultura de responsabilidade compartilhada	A segurança é de toda a equipe de desenvolvimento, incluindo cliente e usuários, não apenas especialista	Pair programming e revisões de código com foco em vulnerabilidades

Adaptado de: Sommerville (2016).

2.5 Retorno de investimento (ROI) no ciclo de entrega ágil

O principal objetivo do desenvolvimento não é apenas cumprir requisitos do software, mas entregar valor de negócio contínuo para o cliente no menor tempo possível. O retorno de investimento (ROI) é observado diretamente na eficácia da aplicação e na sua capacidade de suportar os objetivos estratégicos da organização.

O ROI é maximizado pela adoção de práticas ágeis que garantem que o capital investido se traduza rapidamente em resultados mensuráveis como:

- **Minimizar custo de refatoração e feedbacks rápidos:** consiste na entrega de sprints em ciclos curtos, permitindo que stakeholders validem as funcionalidades de forma rápida, reduzindo, assim, o risco de desenvolver um recurso errado, economizando tempo e recursos.
- **Aumento da produtividade e eficiência operacional:** um software que atende com precisão e clareza aos requisitos de negócio se traduz diretamente em maior velocidade de adoção, redução de erros operacionais e melhoria da satisfação do cliente e usuário.

2.6 Métodos, ferramentas e técnicas (MF&T): melhoria da qualidade de software

Veja a seguir um estudo de caso que destaca os métodos, as ferramentas e as técnicas (MF&T) voltados à melhoria contínua da qualidade de software.

Exemplo de aplicação

Estudo de caso: melhoria da qualidade e produtividade no desenvolvimento ágil de um sistema de gestão hospitalar

A empresa Asserti desenvolve soluções de software para hospitais e clínicas. Seu produto principal, MedicSoft, é um sistema de gestão hospitalar que centraliza dados de pacientes, prontuários eletrônicos e processos administrativos.

Com o crescimento da base de clientes, começaram a surgir problemas de qualidade, como falhas em integrações e lentidão no sistema, impactando diretamente a produtividade das equipes e a satisfação dos usuários.

Durante as últimas três entregas ágeis, a equipe observou:

- aumento de 25% nos defeitos reportados em produção;
- retrabalho elevado, especialmente em módulos de faturamento;
- baixa cobertura de testes automatizados (apenas 35%);
- dificuldade em mensurar a produtividade real das sprints.

A direção decidiu então aplicar uma série de métodos, ferramentas e técnicas para elevar a qualidade do software e otimizar a produtividade da equipe. Foi estruturado um **plano de garantia da qualidade (GQS)**, contemplando:

- padrões de codificação baseados em Cleanroom;
- revisões de código (peer review) obrigatórias via GitHub;
- checklist em conformidade com ISO/IEC 25010 (funcionalidade, confiabilidade, desempenho e segurança);
- auditorias internas de qualidade a cada três sprints.

Métricas de qualidade (ferramentas aplicáveis)

No quadro a seguir são apresentadas algumas das ferramentas aplicáveis para mensurar a evolução da qualidade:

Quadro 5 – Métricas da qualidade e ferramentas aplicáveis

Métrica	Ferramenta/desenvolvedor	Características
Eficiência	Jira/Atlassian	Mede o tempo entre a criação de uma tarefa e sua conclusão Eficiência na remoção de defeitos antes da entrega; permite o acompanhamento de bugs, tarefas e qualidade do processo
Eficiência	GitHub – modo Kanban/ GitHub, Inc. (Microsoft)	Mede o throughput (taxa de conclusão) Tempo médio para corrigir falhas Acompanhamento visual do fluxo de trabalho
Desempenho	GanttProject/ Bartłomiej Nawrocki	Geração de cronogramas, distribuição de responsabilidades e tarefas (ou sprints) Checklist; medição de velocidade da sprint (story points entregues)

Custo da qualidade

Nesse caso, o mapeamento de custos da qualidade se baseou nos quatro componentes:

- **Prevenção:** treinamento, automação de testes e revisões de código.
- **Avaliação:** auditorias, lead time (tempo de entrega total), revisões e inspeções.
- **Falhas internas:** correção de defeitos antes da entrega.
- **Falhas externas:** manutenção corretiva pós-entrega.

O quadro a seguir mostra as principais metas a serem atingidas após seis meses de implementação.

Quadro 6 – Metas a serem atingidas

Categoria	Meta da qualidade	Métrica/medida	Tipo de custo
Eficiência do processo	Reduzir em 20% o tempo médio de resolução de tarefas em três sprints	Lead time	Prevenção e avaliação
Remoção de defeitos antes da entrega	Aumentar em 30% a taxa de correção de defeitos detectados internamente, antes da entrega ao cliente	Taxa de defeitos removidos antes da entrega	Falhas internas
Produtividade da equipe	Elevar a velocidade média da sprint em 10%, mantendo a qualidade estável	Story points entregues/sprint	Avaliação
Confiabilidade pós-entrega	Diminuir em 25% as falhas externas reportadas nos dois primeiros meses após a entrega	Quantidade de bugs reportados pós-entrega	Falhas externas

Conclusão

Com a aplicação de metas de qualidade, há uma melhoria expressiva na eficiência, produtividade e confiabilidade do processo de desenvolvimento. A redução prevista de 20% no lead time deve agilizar entregas e otimizar o fluxo de trabalho.

Espera-se elevar em 30% a correção de defeitos internos, reduzindo falhas externas e custos de manutenção, além de aumentar em 10% a velocidade média das sprints, mantendo a qualidade estável.

A redução estimada de 25% nas falhas pós-entrega indica maior satisfação do cliente e confiabilidade do software. De forma geral, as metas projetam o fortalecimento de uma cultura de prevenção e melhoria contínua, com ganhos em qualidade, custo e previsibilidade.



Lembrete

A melhoria contínua da qualidade de software está diretamente associada à governança em TI, pois ambas visam garantir que a tecnologia entregue valor ao negócio com controle, transparência e alinhamento estratégico.

A governança estabelece diretrizes, papéis, processos e métricas que orientam o desenvolvimento de software, assegurando conformidade com padrões, gestão de riscos e tomada de decisão baseada em indicadores de qualidade. Ao integrar práticas de qualidade como definição de métricas, padronização de processos, auditorias e melhoria contínua à governança em TI, a organização fortalece a confiabilidade dos sistemas, reduz o retrabalho e aumenta a previsibilidade das entregas, promovendo maior eficiência operacional e sustentabilidade dos resultados.



Resumo

A unidade I é composta pelos tópicos "Qualidade e produtividade de software" e "Engenharia de segurança de software".

"Qualidade e produtividade de software" aborda os fundamentos da qualidade e produtividade de software, destacando como a crescente demanda por soluções digitais exige processos cada vez mais eficientes e resultados de alto desempenho. A relação entre qualidade e produtividade é discutida sob a ótica das metodologias ágeis, que priorizam entregas rápidas sem comprometer a confiabilidade.

A seção sobre garantia da qualidade de software (SQA) apresenta os princípios do controle sistemático da qualidade, assegurando que o produto atenda aos RF e RNF. O tópico sobre custo da qualidade diferencia os custos de prevenção, avaliação, falhas internas e externas, ressaltando a importância de investir em ações preventivas para reduzir retrabalho e manutenção.

Os fatores, atributos e métricas da qualidade são explorados como instrumentos essenciais para mensurar, analisar e aprimorar o desempenho do software. Entre eles, destacam-se a eficiência, a confiabilidade, a usabilidade e a manutenibilidade, acompanhadas por métricas específicas aplicáveis em ferramentas como Jira, GitHub e GanttProject.

Já "Engenharia de segurança de software" introduz a engenharia de segurança de software, enfatizando a necessidade de proteger sistemas, dados e processos contra riscos operacionais e vulnerabilidades. O gerenciamento e a análise de riscos, o plano de contingência e o seguro de software são apresentados como práticas essenciais para a continuidade e resiliência do projeto.

A discussão sobre governança de TI evidencia a importância da conformidade, da gestão estratégica e do alinhamento entre tecnologia e objetivos organizacionais. Em seguida, o tema do retorno de investimento (ROI) no ciclo ágil demonstra como a qualidade e a segurança impactam diretamente a sustentabilidade econômica do desenvolvimento.

Por fim, o tópico destaca os métodos, ferramentas e técnicas (MF&T) voltados à melhoria contínua da qualidade de software, integrando automação, métricas e boas práticas de engenharia como pilares para a excelência operacional e a entrega de valor ao cliente.



Exercícios

Questão 1. Uma equipe que desenvolve um sistema web em sprints curtas decidiu "ganhar velocidade" suspendendo as revisões técnicas, reduzindo os testes automatizados e deixando os ajustes de qualidade para "uma etapa final". Após algumas entregas, aumentaram os defeitos na produção e o tempo gasto com correções e refatorações começou a crescer. Considerando a visão sobre a qualidade no desenvolvimento ágil e a garantia da qualidade de software (SQA), qual ação é mais coerente para lidar com o problema?

- A) Manter a estratégia e concentrar a verificação de qualidade em um ciclo posterior, depois que todas as funcionalidades principais estiverem prontas.
- B) Aumentar a entrega de funcionalidades e compensar as falhas com suporte e manutenção corretiva, pois isso reduz o esforço durante o desenvolvimento.
- C) Tratar a qualidade como atividade contínua na sprint, estruturando a SQA com revisões técnicas, controle de configuração/artefatos, testes automatizados e mecanismos de medição e relatórios.
- D) Exigir a "qualidade máxima" com múltiplas rodadas adicionais de revisões formais para cada item, mesmo que isso eventualmente paralise o fluxo de entrega.
- E) Medir produtividade apenas por volume de entregas (exemplo: número de histórias concluídas) e não acompanhar defeitos, pois defeitos são naturais em ciclos curtos.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: no ágil, a qualidade não é deixada para o final; ela ocorre de forma integrada e contínua. "Empurrar a qualidade para depois" reforça o acúmulo de custos futuros.

B) Alternativa incorreta.

Justificativa: abrir mão de práticas de qualidade tende a aumentar os defeitos e o custo de correção posterior, incluindo a manutenção. Essa abordagem não resolve a causa do problema.

C) Alternativa correta.

Justificativa: a SQA consiste em um conjunto sistemático de atividades que acompanha o ciclo de vida e que inclui tarefas como as revisões técnicas e a gestão de configuração, além de testes (automatizados/semiautomatizados), dos mecanismos de medição e dos relatórios. Assim, a implementação da SQA promove a redução de defeitos e ajuda a evitar o acúmulo de débito técnico.

D) Alternativa incorreta.

Justificativa: em projetos reais, deve haver equilíbrio entre a qualidade e a produtividade. O excesso de controles pode reduzir a produtividade em virtude do tempo extra gasto na atividade e não é a abordagem indicada para sustentar entregas incrementais.

E) Alternativa incorreta.

Justificativa: a qualidade relaciona-se aos atributos e aos requisitos (especialmente RNF). O foco exclusivo em entregar rápido pode levar a retrabalho e custos futuros. A prática de ignorar defeitos impede o time de gestão de acompanhar a qualidade de forma objetiva.

Questão 2. Um product owner apresentou os requisitos a seguir para um aplicativo de serviços. O time precisa separar o que é requisito funcional ("o que" o sistema faz) do que é requisito não funcional ("como" o sistema deve operar, ligado à qualidade e às restrições). Qual alternativa contém apenas requisitos não funcionais?

A) "Cadastrar usuário", "gerar relatório mensal", "emitir nota fiscal" e "permitir redefinir senha".

B) "Ter autenticação multifator", "criptografar dados sensíveis", "registrar logs de acesso por 180 dias" e "manter trilha de auditoria para ações críticas, permitindo identificar usuário, data/hora e origem do acesso".

C) "Permitir pagamento por Pix", "tempo de resposta abaixo de 2 segundos", "exportar relatório em PDF" e "tolerar falhas sem interromper o serviço".

D) "Exibir histórico de pedidos", "ser fácil de aprender", "ser compatível com diferentes plataformas" e "notificar o usuário por e-mail".

E) "Gerar relatório semanal", "ter interface atrativa", "rodar em celular" e "permitir anexar documentos".

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: são descrições do que o sistema deve fazer (requisitos funcionais), não propriedades de qualidade nem condições de operação.

B) Alternativa correta.

Justificativa: trata-se de requisitos não funcionais ligados à segurança e à auditabilidade/rastreabilidade (mecanismos de controle de acesso, de proteção de dados e registro de eventos). São requisitos que especificam restrições e atributos de qualidade sobre o funcionamento do sistema, isto é, descrevem como ele deve operar para reduzir os riscos e permitir a verificação de ações.

